# Validación preliminar del Quantum Analysis Framework (QAF) mediante benchmarking computacional reproducible

Comparación de reglas greedy, MILP, QUBO con recocido simulado, inferencia contextual y planificación probabilística bajo incertidumbre

Juan Manuel Goenaga

Septiembre de 2026

**Resumen**

El objetivo de este trabajo es **validar experimentalmente, de forma preliminar y falsable, una hipótesis central del Quantum Analysis Framework (QAF)**: que una representación contextual de relaciones entre decisiones puede mejorar la calidad de asignación de recursos bajo información incierta y señales potencialmente engañosas, en comparación con formulaciones convencionales que tratan las amenazas de forma principalmente marginal. Se construyó en Python un banco de pruebas sintético y reproducible para un problema de asignación de $N$ amenazas a $M$ recursos, con $K = N \times M$ variables binarias, escalando desde $K = 15$ hasta $K = 500$. Se compararon siete configuraciones: Greedy, MILP/HiGHS, QUBO con recocido simulado, MILP con actualización contextual, un proxy QAF-QUBO con el mismo solver de recocido simulado, un híbrido BQM local y un planificador probabilístico por partículas inspirado en POMDP. Se evaluaron 90 escenarios independientes y 630 ejecuciones de métodos, con tres niveles de incertidumbre.

El proxy QAF-QUBO alcanzó en promedio 89.71 % del daño óptimo evitable, frente a 87.62 % del QUBO convencional con el mismo solver, equivalente a una mejora relativa de 2.38 %. Más importante, redujo la tasa de recursos asignados a señuelos de 13.26 % a 6.11 %, una reducción relativa de 53.91 %. La transformación contextual también redujo el Brier score de las creencias de 0.1243 a 0.1079, una mejora de calibración del 13.22 %. Sin embargo, el beneficio no fue uniforme: con incertidumbre alta (60 %) el proxy QAF-QUBO quedó aproximadamente 4 % por debajo de MILP y QUBO convencional, mostrando que la intensidad del término contextual requiere calibración. Asimismo, un MILP alimentado con la misma actualización contextual superó al proxy QAF-QUBO, y el planificador probabilístico por partículas fue el mejor método agregado. Por tanto, los resultados **apoyan la hipótesis de que la información contextual aporta valor**, pero **no validan todavía la superioridad general del QAF completo ni una ventaja computacional end-to-end**. El principal resultado científico es la identificación del siguiente problema de investigación: aislar el valor de la representación QAF de la calidad del solver y probar la dinámica MPS/TDVP completa frente a baselines probabilísticos y de optimización fuertes.

**Palabras clave:** Quantum Analysis Framework, QAF, QUBO, benchmarking, incertidumbre, POMDP, MILP, recocido simulado, decisiones críticas, redes tensoriales.

# 1. Introducción

El Quantum Analysis Framework (QAF) ha sido propuesto como una arquitectura para representar decisiones complejas bajo incertidumbre mediante estructuras quantum-like, dinámica no conmutativa, redes tensoriales y compilación hacia problemas QUBO. El manuscrito técnico de referencia plantea una arquitectura basada en MPS/QTT, TDVP, matrices de densidad reducida y compilación asíncrona QUBO, y reporta una prueba de referencia en Julia para $K = 500$ variables. Sin embargo, una evaluación rigurosa requiere separar cuatro preguntas: (i) si la representación contextual mejora la estimación del estado; (ii) si mejora la calidad de la decisión; (iii) si esa mejora persiste al aumentar incertidumbre y escala; y (iv) si el coste computacional total es competitivo frente a métodos establecidos.

La arquitectura quantum-like multiagente complementaria propone, además, medir confianza, detectar contradicciones, decidir cuándo consultar información adicional y escalar a supervisión humana. Esta formulación transforma la promesa de QAF en una proposición experimental: la incertidumbre y las relaciones contextuales deben producir diferencias medibles en calibración, uso de recursos, robustez y tiempo de decisión.

Este trabajo no intenta demostrar una "ventaja cuántica" ni reproducir todavía la totalidad de MPS/TDVP. En cambio, implementa un **proxy QAF contextual** deliberadamente limitado para probar un supuesto específico y construir un banco de benchmarking reutilizable. La transparencia sobre esta delimitación es esencial: una mejora del proxy contextual es evidencia sobre la hipótesis de representación, no sobre toda la arquitectura QAF.

# 2. Pregunta de investigación e hipótesis

La pregunta central es:

> **¿Una formulación contextual inspirada en QAF puede producir decisiones de asignación de recursos de mayor calidad y menor desperdicio ante señales ambiguas o correlacionadas que baselines convencionales, manteniendo escalabilidad computacional hasta $K = 500$ variables?**

Se definieron cuatro hipótesis falsables:

- **H1 - Calibración contextual:** la actualización contextual reduce el error de confianza (Brier score) frente a la fusión convencional de sensores.
- **H2 - Calidad con solver controlado:** QAF-QUBO usando el mismo recocido simulado que el QUBO convencional obtiene mayor calidad de decisión o menor desperdicio de recursos.
- **H3 - Robustez:** la ventaja contextual se mantiene o aumenta cuando crece la incertidumbre.
- **H4 - Eficiencia end-to-end:** el pipeline QAF-QUBO mantiene latencia compatible con una ventana de 500 ms y muestra una trayectoria de escalabilidad competitiva.

# 3. Estado del arte del benchmarking

## 3.1. Greedy y reglas

Los algoritmos greedy ordenan alternativas por una función local de valor y asignan secuencialmente mientras respetan restricciones. Constituyen un baseline indispensable porque son extremadamente rápidos y explicables, aunque pueden ignorar interacciones globales y valor de información.

## 3.2. MILP/MIQP

La optimización entera mixta es un baseline clásico fuerte para problemas de asignación. En esta prueba se utiliza `scipy.optimize.milp`, una interfaz a HiGHS que resuelve objetivos lineales bajo restricciones lineales e integridad. La documentación de SciPy describe el solver como determinista y orientado a encontrar el óptimo global en MILP moderadamente desafiantes. En problemas cuadráticos, herramientas MIQP comerciales o especializadas extienden esta familia.

## 3.3. QUBO y recocido simulado

Un problema QUBO minimiza $x^T Qx$ para $x \in \{0,1\}^K$. Es una formulación natural para optimización binaria, y puede resolverse mediante heurísticas clásicas, annealing cuántico o métodos híbridos. Para aislar el efecto del modelo, el presente estudio usa el **mismo solver de recocido simulado** para QUBO convencional y QAF-QUBO.

## 3.4. POMDP y POMCP

Los POMDP representan explícitamente estados no observables y creencias condicionadas a observaciones. POMCP, introducido por Silver y Veness, combina actualización Monte Carlo de la creencia con Monte Carlo Tree Search y demostró escalabilidad a espacios de estado extremadamente grandes. El presente estudio no implementa POMCP completo; usa un planificador por partículas de horizonte único como baseline probabilístico intermedio. Esta limitación se conserva explícitamente para evitar atribuir al benchmark capacidades que no fueron implementadas.

## 3.5. D-Wave Hybrid BQM

D-Wave ofrece solvers híbridos para Binary Quadratic Models (BQM). La documentación actual de Leap describe un servicio híbrido quantum-classical que acepta BQM y reporta, para el solver BQM vigente, hasta un millón de variables, con un tiempo mínimo de servicio de 3 s hasta 1024 variables. Estas cifras no son directamente comparables con tiempos locales de compilación o solución, porque incluyen una arquitectura de servicio diferente. El código adjunto incluye un adaptador para `LeapHybridSampler`, pero no se ejecutó en este experimento por ausencia de credenciales Leap en el entorno de ejecución. En su lugar se incluyó un híbrido BQM local como surrogate metodológico, claramente identificado como no-QPU.

# 4. Materiales y métodos

## 4.1. Problema de asignación

Se modelan $N$ amenazas y $M$ recursos. Cada variable binaria

$$x_{ir} \in \{0, 1\}$$

indica si el recurso $r$ es asignado a la amenaza $i$. El tamaño del espacio de decisión es

$$K = N \times M.$$

Cada amenaza posee un daño potencial $d_i$, un estado real oculto $z_i \in \{0, 1\}$ (amenaza real o señuelo), y una confianza convencional $p_i$ obtenida por fusión de tres sensores sintéticos. Cada recurso tiene una efectividad $e_{ir}$ y un coste $c_r$.

La utilidad esperada convencional por asignación es

$$V_{ir} = p_i d_i e_{ir} - c_r.$$

Se impone como máximo una asignación por amenaza y una por recurso.

## 4.2. Incertidumbre y señuelos

La incertidumbre controla tres fenómenos del generador sintético: solapamiento entre distribuciones sensoras de amenazas reales y señuelos, pérdida de observaciones y proporción de señuelos. Un subconjunto de señuelos es deliberadamente *spoofed*: recibe lecturas sensoras semejantes a una amenaza real. Además, el generador crea relaciones contextuales negativas entre un señuelo spoofed y una amenaza real de alta confianza, representando un caso simplificado de firmas mutuamente incompatibles o correlacionadas.

## 4.3. Proxy QAF contextual

El proxy QAF modifica la creencia marginal usando el grafo contextual. Para pares negativos, el estado con mayor confianza es reforzado y el más débil es suprimido; para relaciones positivas se introduce soporte moderado. La actualización se aplica en el espacio logit:

$$\tilde{p}_i = \sigma\left(\mathrm{logit}(p_i) + \gamma_i C_i\right),$$

donde $C_i$ resume las relaciones contextuales normalizadas. El QUBO contextual añade además una penalización a la selección simultánea de amenazas con correlación negativa fuerte.

Esta formulación es un **proxy experimental**; no implementa la evolución MPS/TDVP ni las 1-RDM/2-RDM completas del QAF teórico. Su función es probar si la capa de contexto propuesta por QAF contiene señal útil antes de invertir en una implementación completa.

## 4.4. Métodos comparados

| Método | Qué prueba | Implementación |
|---|---|---|
| Greedy | Baseline de reglas rápidas | Orden descendente de valor esperado, respetando unicidad de amenaza y recurso. |
| MILP | Optimización clásica fuerte | SciPy/HiGHS con variables binarias y restricciones de asignación. |
| QUBO-SA | Baseline QUBO convencional | QUBO de valor esperado + penalizaciones; recocido simulado JIT con Numba. |
| Context-MILP | Ablación de la representación | Usa la creencia contextual QAF pero optimiza con MILP, separando representación de solver. |
| QAF-QUBO-SA | Hipótesis QAF proxy | Creencia contextual + acoplamientos negativos + mismo SA del QUBO convencional. |
| Hybrid-BQM-local | Surrogate híbrido | Semilla greedy + multi-start SA; no es D-Wave remoto ni usa QPU. |
| Particle-POMDP | Baseline probabilístico | Muestreo de partículas con exclusividad contextual y asignación MILP; no es POMCP completo. |

Cuadro 1: Métodos implementados en el banco de pruebas.

## 4.5. Diseño experimental

Se probaron cinco escalas: $(N, M) = (5, 3), (10, 5), (20, 5), (25, 10), (50, 10)$, correspondientes a $K = 15, 50, 100, 250, 500$. Para cada escala se utilizaron niveles de incertidumbre 0.10, 0.35 y 0.60, con seis semillas independientes por combinación. Esto produjo 90 escenarios y 630 ejecuciones de métodos.

El *ground truth* se reservó únicamente para evaluación. La calidad se expresa como porcentaje del daño evitado por un oracle MILP que conoce $z_i$. Otras métricas fueron desperdicio en señuelos, latencia, tamaño de representación, cumplimiento de 500 ms y Brier score de la confianza.

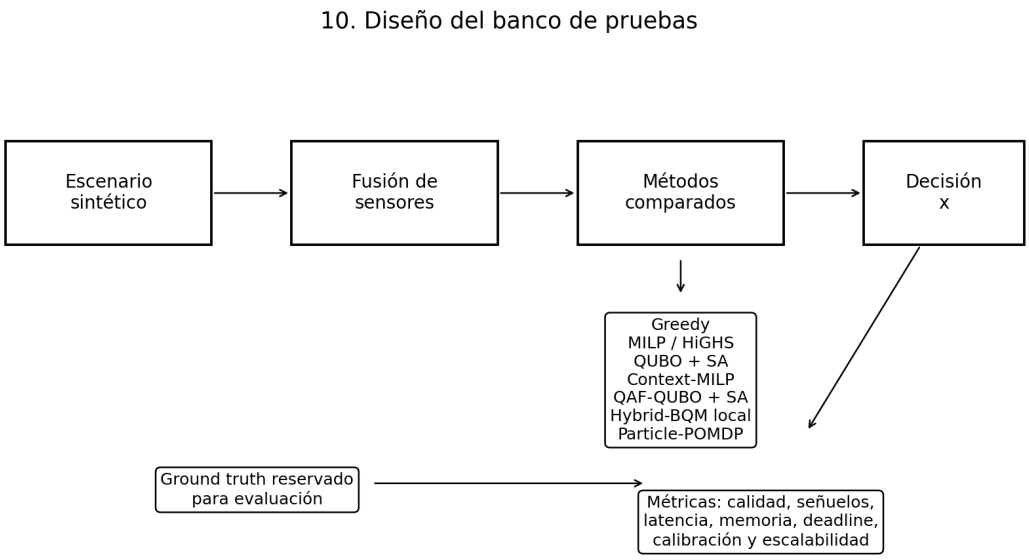



Figura 1: Diseño del banco de pruebas y separación entre información disponible para los métodos y ground truth reservado para evaluación.

# 5. Resultados

## 5.1. Resultado agregado

| **Método** | **Calidad %** | **Señuelos %** | **Latencia med. ms** | **Deadline %** |
|---|---|---|---|---|
| Greedy | 88.28 | 12.30 | 0.06 | 100 |
| MILP | 88.71 | 12.93 | 4.59 | 100 |
| QUBO-SA | 87.62 | 13.26 | 17.49 | 100 |
| Context-MILP | 92.49 | 8.59 | 4.50 | 100 |
| QAF-QUBO-SA | 89.71 | 6.11 | 18.39 | 100 |
| Hybrid-BQM-local | 87.62 | 13.26 | 18.30 | 100 |
| Particle-POMDP | 95.16 | 3.70 | 6.25 | 100 |

Cuadro 2: Resultados agregados sobre 90 escenarios sintéticos. Calidad = porcentaje del daño óptimo evitable.

Comparado con QUBO-SA usando el mismo solver, QAF-QUBO-SA incrementó la calidad media de 87.62 % a 89.71 %, equivalente a una mejora relativa de 2.38 %, y redujo el desperdicio en señuelos de 13.26 % a 6.11 %, una reducción relativa de 53.91 %. Frente a MILP, el proxy QAF mostró una mejora relativa de 1.13 % en calidad y 52.72 % de reducción de desperdicio, pero con aproximadamente cuatro veces la latencia mediana.

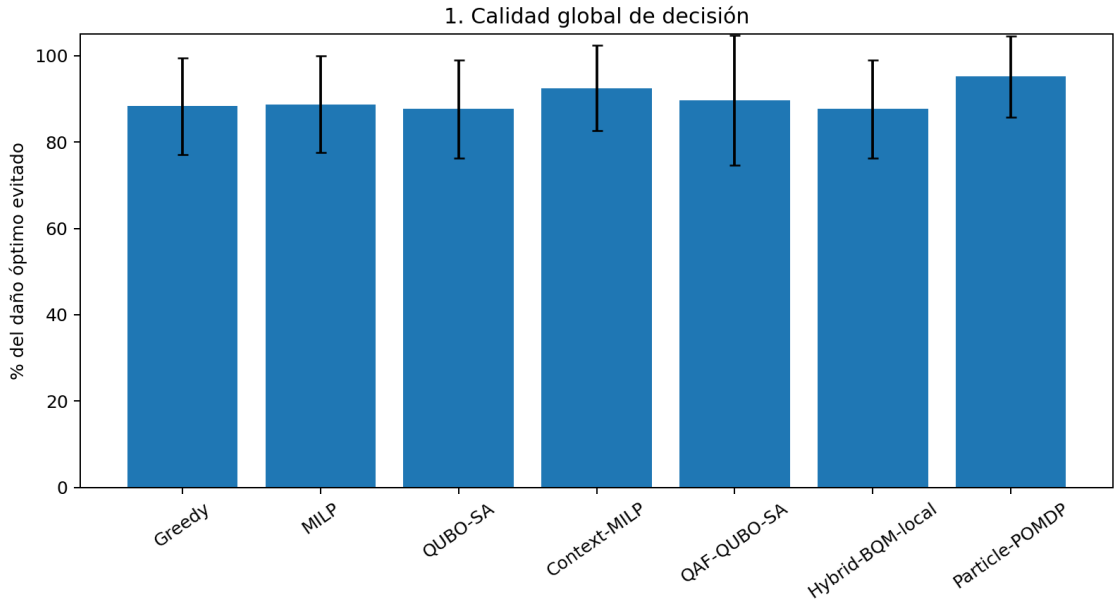



Figura 2: Calidad global de decisión. Las barras de error muestran desviación estándar entre escenarios.

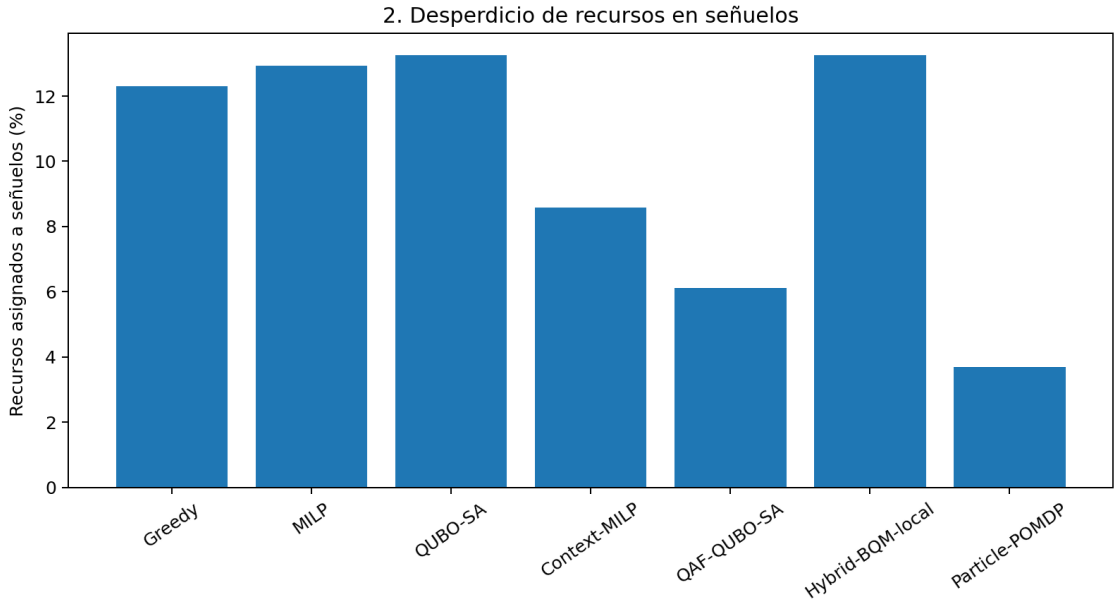



Figura 3: Proporción de recursos asignados a señuelos. Menor es mejor.

## 5.2. La información contextual aporta valor, independientemente del solver

Context-MILP alcanzó 92.49 % de calidad frente a 88.71 % de MILP convencional, una mejora relativa de 4.26 %, y redujo el desperdicio en señuelos en 33.52 %. Como ambos usan el mismo optimizador, esta diferencia apoya H1: la actualización contextual contiene información útil en el generador sintético.

La calibración refuerza esta lectura. El Brier score promedio disminuyó de 0.1243 a 0.1079, una reduc-

ción de 13.22 %. La mejora de Brier fue 19.70 % con incertidumbre baja, 15.42 % con incertidumbre media y 8.75 % con incertidumbre alta.

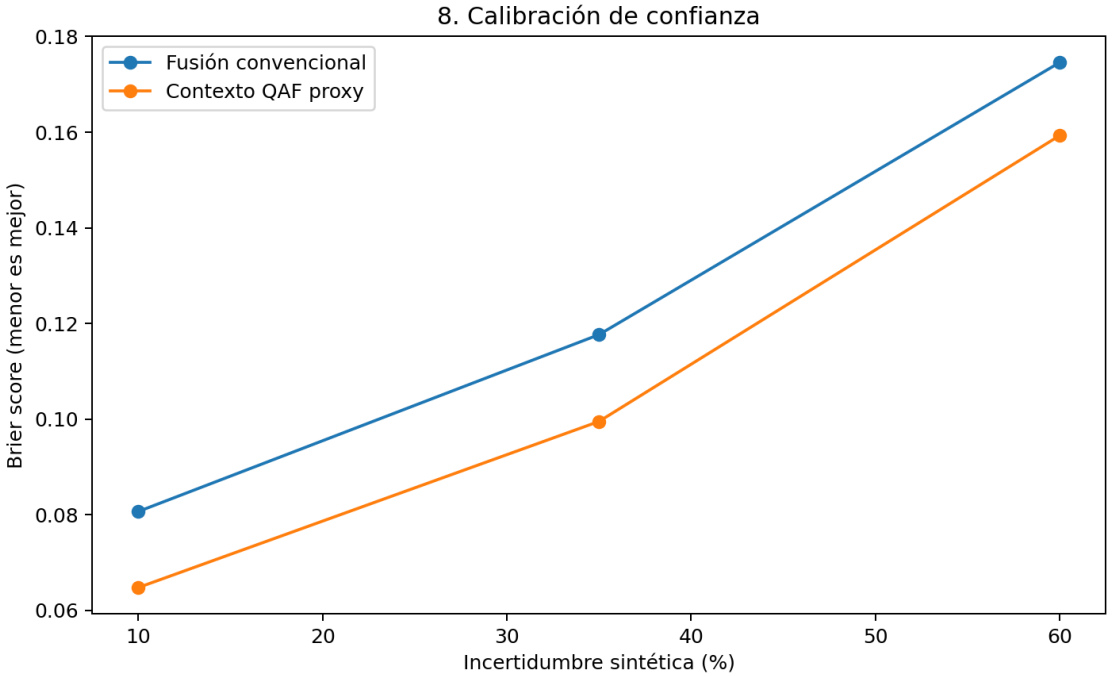



Figura 4: Brier score de la confianza convencional y de la actualización contextual QAF proxy.

### 5.3. La ventaja QAF no es monótona con la incertidumbre

Con incertidumbre 10 %, QAF-QUBO-SA obtuvo 97.77 % de calidad, 4.21 % mejor que MILP y 5.70 % mejor que QUBO-SA en términos relativos. Con incertidumbre 35 %, la ventaja fue 2.92 % frente a MILP y 5.11 % frente a QUBO-SA. Sin embargo, con incertidumbre 60 %, QAF-QUBO-SA cayó a 81.67 %, aproximadamente 4.11 % por debajo de MILP y 3.97 % por debajo de QUBO-SA.

Este resultado falsifica una versión fuerte de H3: con la calibración utilizada, el término contextual se vuelve demasiado agresivo cuando las señales marginales son poco informativas. Esto no elimina la hipótesis QAF, pero identifica un requisito de diseño: $\gamma$ debe ser adaptativo a la confiabilidad y densidad de evidencia, no fijo.

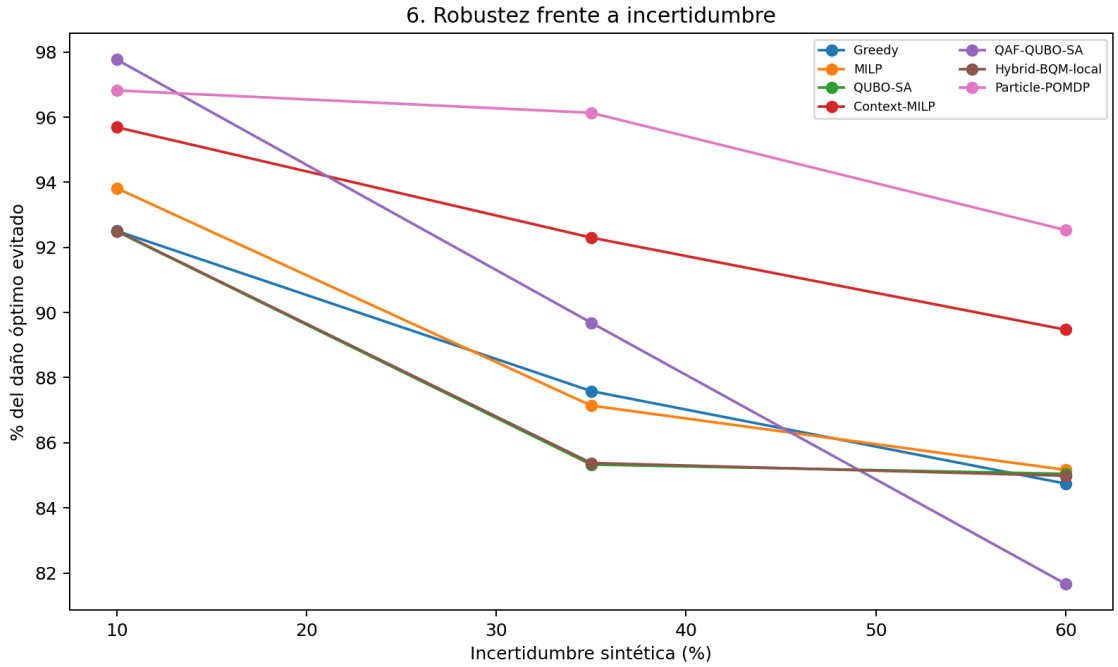



Figura 5: Calidad de decisión frente al nivel de incertidumbre sintética.

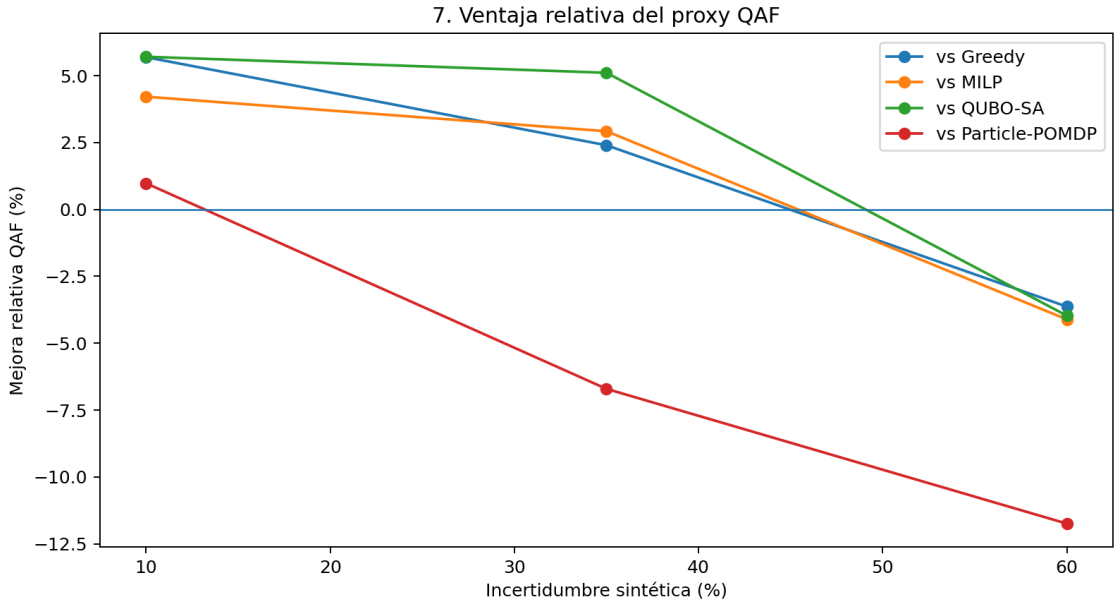



Figura 6: Mejora relativa del proxy QAF frente a baselines seleccionados. Los valores negativos indican pérdida de calidad.

## 5.4. Escalabilidad temporal

Todos los métodos completaron el benchmark local por debajo de 500 ms en las instancias hasta $K = 500$. Por tanto, el umbral de 500 ms no fue discriminante en esta prueba. En $K = 500$, la latencia mediana fue 0.14 ms para Greedy, 17.52 ms para MILP, 281.88 ms para QUBO-SA y 302.88 ms para QAF-QUBO-SA. El planificador por partículas requirió 23.68 ms.

Esto conduce a una conclusión importante respecto de la afirmación previa de "18.5 ms para $K = 500$": una cifra de compilación no debe interpretarse como latencia end-to-end. En el presente benchmark, la etapa de búsqueda por SA domina el tiempo QAF-QUBO. La validación futura debe reportar por separado $T_{estado}$, $T_{compilacion}$, $T_{solver}$ y $T_{decodificacion}$.

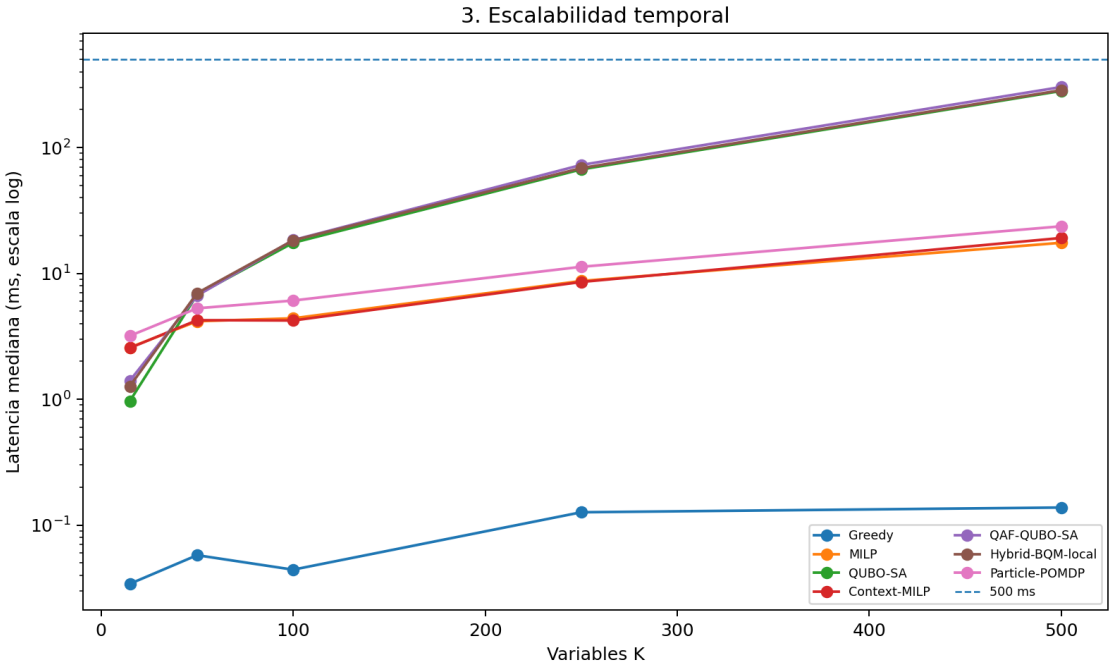



Figura 7: Latencia mediana por escala. El eje vertical es logarítmico.

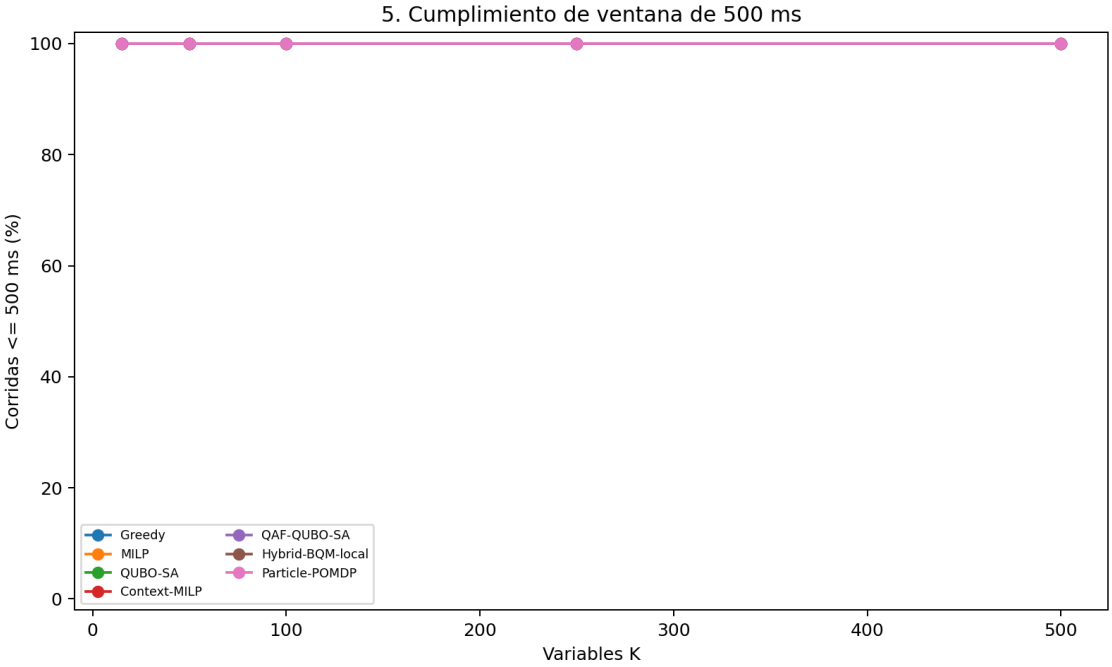



Figura 8: Porcentaje de ejecuciones dentro de la ventana de 500 ms. En este rango todas las configuraciones cumplieron el umbral.

## 5.5. Escalabilidad de representación

El modelo QUBO explícito crece por sus acoplamientos, mientras Greedy y MILP lineal mantienen representaciones más compactas para esta formulación. En $K = 500$, el tamaño de arrays del QAF-QUBO proxy fue aproximadamente 360 KB, frente a 344 KB del QUBO estándar y 19.5 KB del modelo lineal utilizado por MILP. Estas cifras miden las estructuras explícitas del benchmark, no memoria RSS completa ni la memoria de una futura MPS comprimida.

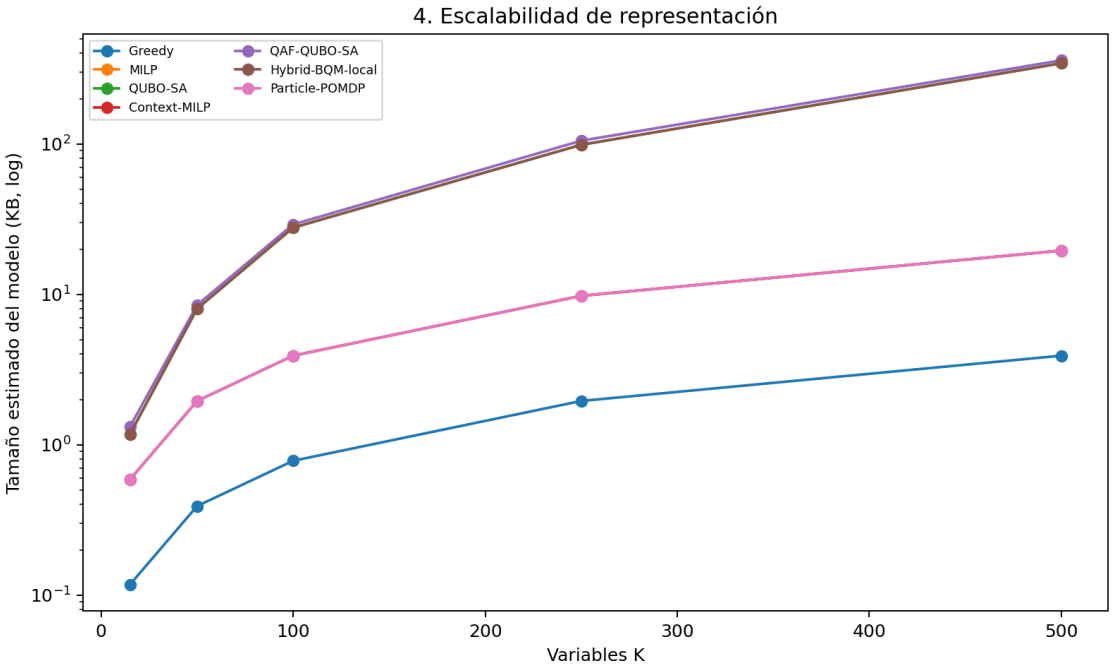



Figura 9: Tamaño estimado de las estructuras explícitas del modelo.

## 5.6. Frontera calidad-latencia en $K = 500$

En la mayor escala, QAF-QUBO-SA mejoró modestamente la calidad frente al QUBO convencional (87.22 % vs. 85.42 %) y redujo el desperdicio de señuelos (7.78 % vs. 13.89 %), pero fue más lento. Context-MILP obtuvo 92.01 % con 19.12 ms y Particle-POMDP 96.15 % con 23.68 ms. Esto indica que, en este proxy, la mayor oportunidad está en la representación contextual y no en el solver SA utilizado.

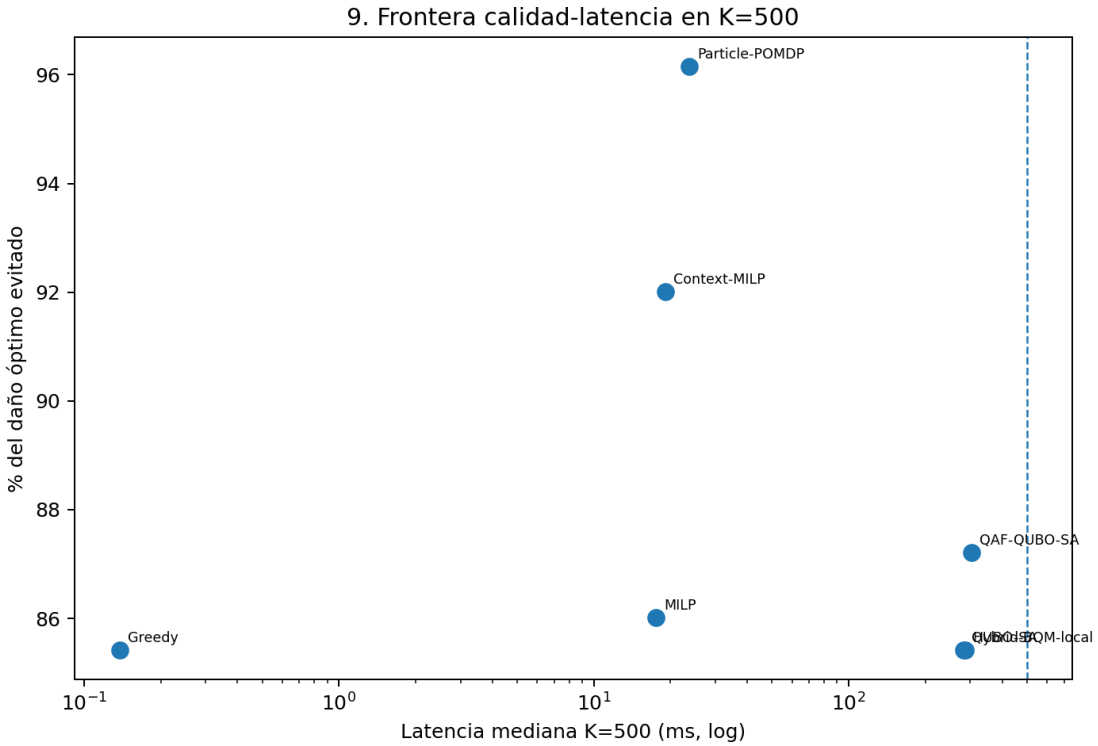



Figura 10: Frontera calidad-latencia para $K = 500$.

## 6. Sabidurías halladas

1. **El contexto sí contiene señal útil en el experimento.** La mejora de Context-MILP frente a MILP y la reducción del Brier score apoyan la hipótesis de que relaciones entre amenazas pueden mejorar la inferencia respecto de tratar cada señal de forma independiente.

2. **El mayor beneficio del proxy QAF apareció en el manejo de señuelos, no en velocidad.** QAF-QUBO-SA redujo aproximadamente 54 % el desperdicio frente a QUBO-SA con el mismo solver.

3. **El solver importa tanto como la representación.** Context-MILP superó a QAF-QUBO-SA. Por tanto, atribuir toda mejora a "QAF" sin controlar el optimizador sería metodológicamente incorrecto.

4. **La incertidumbre extrema expone una debilidad de calibración.** El proxy contextual con ganancia fija se sobrecorrige a 60 % de incertidumbre. La próxima versión debe aprender o adaptar la fuerza contextual a la confiabilidad de las observaciones.

5. **Un baseline probabilístico fuerte es obligatorio.** Particle-POMDP fue el mejor agregado. Una validación DARPA seria debe incluir POMCP completo o un planner bayesiano comparable, no solo Greedy y SA.

6. **Compilación y decisión no son la misma métrica.** La latencia de construcción de un QUBO debe separarse de la latencia de resolución. Una afirmación de milisegundos de compilación no implica una decisión end-to-end de igual duración.

7. **El benchmark actual valida una hipótesis parcial, no el QAF completo.** Para validar MPS/TDVP será necesario reproducir evolución temporal, 1-RDM/2-RDM, crecimiento de dimensión de enlace $\chi$, ablaciones y comparadores bajo cambios dinámicos del estado.

# 7. Evaluación de las hipótesis

| Hipótesis | Resultado | Interpretación |
|---|---|---|
| H1 Calibración contextual | Apoyada en el modelo sintético | Brier -13.22 %; Context-MILP +4.26 % vs. MILP. |
| H2 QAF con solver controlado | Apoyo parcial | QAF-QUBO-SA +2.38 % en calidad y -53.91 % en desperdicio vs. QUBO-SA. |
| H3 Robustez creciente | No apoyada en su forma fuerte | La ventaja desaparece a 60 % de incertidumbre; requiere calibración adaptativa. |
| H4 Eficiencia end-to-end | Parcial | Cumple 500 ms hasta $K = 500$, pero no supera MILP ni el planner por partículas en latencia. |

Cuadro 3: Resultado de falsación/apoyo de las hipótesis preliminares.

# 8. Limitaciones

Este estudio utiliza datos sintéticos y un proxy contextual. No utiliza telemetría real de defensa, no ejecuta hardware cuántico, no implementa TDVP/MPS completo y no implementa POMCP con árbol de observaciones. El resultado depende de la forma en que el generador introduce señuelos y relaciones negativas. Por tanto, los porcentajes no deben extrapolarse a escenarios operacionales reales.

El baseline D-Wave remoto tampoco fue ejecutado: el código contiene un adaptador compatible con `LeapHybridSampler`, pero el entorno no dispone de credenciales Leap. La documentación actual del servicio se usa únicamente como referencia de estado del arte, no como resultado medido.

Finalmente, la métrica de tamaño de modelo corresponde a arrays explícitos del prototipo; no es una medición exhaustiva de memoria de proceso. Una comparación de memoria de MPS deberá instrumentar RSS y perfiles de asignación en una implementación tensorial real.

# 9. Próximos experimentos

La siguiente fase debería:

1. implementar MPS/TDVP real en Python o Julia y medir separadamente compilación, evolución y solve;
2. introducir densidad de acoplamiento como eje experimental adicional;

3. ejecutar ablaciones: sin contexto, solo contexto marginal, contexto + acoplamientos, MPS sin TDVP, QAF completo;
4. reemplazar Particle-POMDP por POMCP completo en un entorno dinámico con observaciones sucesivas;
5. ejecutar QUBO estándar y QAF-QUBO sobre el mismo backend D-Wave Hybrid y, si es posible, QPU/annealer;
6. calibrar $\gamma$ de forma adaptativa usando confiabilidad y entropía de evidencia;
7. preregistrar criterios de éxito y fracaso antes de nuevos experimentos.

## 10. Conclusiones

El benchmark proporciona una **validación preliminar y limitada** de una parte de la propuesta QAF: las relaciones contextuales pueden mejorar la calibración y reducir errores de asignación a señuelos en un entorno sintético. Manteniendo constante el solver SA, el proxy QAF-QUBO obtuvo una mejora relativa de 2.38 % en calidad y una reducción de 53.91 % en desperdicio respecto del QUBO convencional. Sin embargo, no se observó superioridad universal: a alta incertidumbre el proxy perdió rendimiento, y baselines contextuales/probabilísticos convencionales obtuvieron mejores resultados agregados.

Por ello, la conclusión científica no es que "QAF ya supera el estado del arte". La conclusión más sólida es que **existe una señal experimental suficiente para justificar una segunda fase de validación, pero el avance debe aislarse mejor**. El próximo objetivo debe demostrar si la dinámica MPS/TDVP añade una ventaja adicional sobre la actualización contextual clásica y sobre planners probabilísticos fuertes, bajo un protocolo predefinido, reproducible y end-to-end.

## Referencias

1. Bancells, A. (2026). *Quantum Analysis Framework (QAF): dinámica operacional no conmutativa y compilación QUBO asíncrona para la toma de decisiones complejas bajo incertidumbre.* Manuscrito técnico.
2. Rojas Rodríguez, C. D. (2026). *Arquitectura quantum-like multiagente para apoyo a decisiones críticas en entornos de defensa bajo incertidumbre.* UNIR / ParkiApp Research.
3. Silver, D., & Veness, J. (2010). Monte-Carlo Planning in Large POMDPs. *Advances in Neural Information Processing Systems 23.* https://papers.nips.cc/paper/2010/hash/edfbe1afcf9246bb0d40eb4d8 html
4. SciPy Developers. *scipy.optimize.milp - Mixed-integer linear programming.* Consultado en 2026. https://docs.scipy.org/doc/scipy/reference/generated/scipy.optimize.milp.html
5. D-Wave Systems. *BQM Solver Properties.* Consultado en 2026. https://docs.dwavequantum.com/en/latest/industrial_optimization/solver_bqm_properties.html

6. D-Wave Systems. *LeapHybridSampler.sample*. Consultado en 2026. https://docs.dwavequantum.com/en/latest/ocean/api_ref_system/generated/dwave.system.samplers.LeapHybridSampler.sample.html

7. Lucas, A. (2014). Ising formulations of many NP problems. *Frontiers in Physics*, 2, 5.

8. Schollwöck, U. (2011). The density-matrix renormalization group in the age of matrix product states. *Annals of Physics*, 326(1), 96–192.

9. Oseledets, I. V. (2011). Tensor-Train Decomposition. *SIAM Journal on Scientific Computing*, 33(5), 2295–2317.

10. Haegeman, J. et al. (2016). Unifying time evolution and optimization with matrix product states. *Physical Review B*, 94, 165116.

11. Yukalov, V. I., & Sornette, D. (2008). Quantum decision theory as quantum theory of measurement. *Physics Letters A*, 372, 6867–6871.

## A. Reproducibilidad

El archivo `qaf_benchmark.py` incluido con este manuscrito genera los escenarios, ejecuta los métodos, guarda los resultados CSV y produce las diez figuras. Ejecución utilizada:

```
python qaf_benchmark.py --outdir /mnt/data/qaf_benchmark --reps 6
```

## B. Código Python completo

```
#!/usr/bin/env python3
"""
Preliminary benchmark for QAF validation.

IMPORTANT SCIENTIFIC SCOPE
--------------------------
This code does NOT implement the full MPS/TDVP QAF stack described in the QAF paper.
It implements a QAF-inspired contextual QUBO proxy designed to test one narrow
hypothesis: whether contextual pair relationships (including mutually exclusive /
negative correlations suggestive of decoys) improve allocation quality under
uncertainty when the downstream optimizer is held fixed.

Baselines:
  1. Greedy rules
  2. MILP (SciPy/HiGHS)
  3. Standard QUBO + local simulated annealing
  4. Context-aware MILP ablation (same contextual marginal beliefs as QAF proxy)
  5. QAF-inspired contextual QUBO + same simulated annealing
  6. Hybrid-BQM local surrogate (greedy seed + multi-start SA + local repair)

A D-Wave Leap Hybrid adapter is included but is only executed when Ocean is
installed and a configured Leap credential is available. It is not required for
this local benchmark.

The benchmark uses synthetic scenarios; results are evidence about this synthetic
```

```
model, not operational defense performance.
"""

from __future__ import annotations

import argparse
import json
import math
import os
import time
import tracemalloc
from dataclasses import dataclass
from pathlib import Path
from typing import Dict, List, Tuple, Optional

import numpy as np
import pandas as pd
from scipy.optimize import milp, Bounds, LinearConstraint
from scipy.sparse import csr_matrix
from numba import njit

# ------------------------------
# Scenario model
# ------------------------------

@dataclass
class Scenario:
    N: int
    M: int
    uncertainty: float
    seed: int
    truth: np.ndarray # (N,) 1 real, 0 decoy
    damage: np.ndarray # (N,)
    effectiveness: np.ndarray # (N,M)
    resource_cost: np.ndarray # (M,)
    p_base: np.ndarray # (N,) conventional fused confidence
    corr: np.ndarray # (N,N), contextual pair relation in [-1,1]
    spoofed: np.ndarray # (N,) bool

    @property
    def K(self) -> int:
        return self.N * self.M


def _sigmoid(x):
    return 1.0 / (1.0 + np.exp(-x))


def _logit(p):
    p = np.clip(p, 1e-5, 1 - 1e-5)
    return np.log(p / (1 - p))


def generate_scenario(N: int, M: int, uncertainty: float, seed: int) -> Scenario:
    """Generate a synthetic threat/resource allocation scenario.

    The generator intentionally contains adversarial/spoofed decoys. Some spoofed
    decoys are strongly negatively related to a higher-confidence real threat.
    This creates a controlled test for contextual reasoning.
    """
    rng = np.random.default_rng(seed)
    uncertainty = float(np.clip(uncertainty, 0.0, 0.9))
```

```
88
89      decoy_rate = 0.20 + 0.25 * uncertainty
90      truth = (rng.random(N) > decoy_rate).astype(np.int8)
91      # Ensure both classes and enough real threats to make allocation meaningful.
92      if truth.sum() < max(1, min(M, N // 2)):
93          truth[np.argsort(rng.random(N))[: max(1, min(M, N // 2))]] = 1
94      if truth.sum() == N and N >= 3:
95          truth[rng.integers(0, N)] = 0
96
97      # Latent damage if the signal were a real threat. Decoys can look dangerous.
98      damage = rng.uniform(0.30, 1.00, N)
99      effectiveness = rng.uniform(0.55, 0.97, (N, M))
100     resource_cost = rng.uniform(0.025, 0.080, M)
101
102     # Three synthetic sensors. Uncertainty reduces class separation and increases missingness.
103     sensor_weights = np.array([0.42, 0.34, 0.24])
104     scores = np.zeros((N, 3), dtype=float)
105     missing = rng.random((N, 3)) < (0.03 + 0.22 * uncertainty)
106
107     # Base means compress toward 0.5 as uncertainty increases.
108     real_mean = 0.88 - 0.30 * uncertainty
109     decoy_mean = 0.18 + 0.28 * uncertainty
110     concentration = max(4.0, 18.0 * (1.0 - 0.65 * uncertainty))
111
112     for i in range(N):
113         mean = real_mean if truth[i] else decoy_mean
114         a = max(0.5, mean * concentration)
115         b = max(0.5, (1 - mean) * concentration)
116         scores[i] = rng.beta(a, b, 3)
117
118     # Spoof a subset of decoys so their raw sensor confidence resembles a real threat.
119     decoys = np.where(truth == 0)[0]
120     spoofed = np.zeros(N, dtype=bool)
121     if len(decoys):
122         n_spoof = max(1, int(round(len(decoys) * (0.35 + 0.35 * uncertainty))))
123         spoof_idx = rng.choice(decoys, size=min(n_spoof, len(decoys)), replace=False)
124         spoofed[spoof_idx] = True
125         spoof_mean = 0.72 - 0.08 * uncertainty
126         a = spoof_mean * concentration
127         b = (1 - spoof_mean) * concentration
128         scores[spoof_idx] = rng.beta(max(a, .5), max(b, .5), (len(spoof_idx), 3))
129
130     scores[missing] = np.nan
131     weighted_num = np.nansum(scores * sensor_weights[None, :], axis=1)
132     weighted_den = np.nansum((~np.isnan(scores)) * sensor_weights[None, :], axis=1)
133     p_base = np.divide(weighted_num, weighted_den, out=np.full(N, 0.5), where=weighted_den > 0)
134     p_base = np.clip(p_base, 0.02, 0.98)
135
136     # Context graph. Negative links connect spoofed decoys to stronger real candidates.
137     corr = np.zeros((N, N), dtype=float)
138     real_idx = np.where(truth == 1)[0]
139     for i in np.where(spoofed)[0]:
140         if len(real_idx):
141             # Pair with a high-confidence real threat if available.
142             j = real_idx[np.argmax(p_base[real_idx] * damage[real_idx])]
143             if i != j:
144                 c = -rng.uniform(0.65, 0.95)
145                 corr[i, j] = corr[j, i] = c
146
147     # Add a sparse set of positive contextual relationships between real threats.
148     if len(real_idx) >= 2:
149         n_pos = max(1, int(0.05 * N))
```

```
        for _ in range(n_pos):
            i, j = rng.choice(real_idx, size=2, replace=False)
            c = rng.uniform(0.20, 0.55)
            corr[i, j] = corr[j, i] = max(corr[i, j], c)

    return Scenario(N, M, uncertainty, seed, truth, damage, effectiveness,
                    resource_cost, p_base, corr, spoofed)


def qaf_context_belief(s: Scenario, gamma_neg: float = 4.0, gamma_pos: float = 0.8) -> np.ndarray:
    """QAF-inspired contextual belief proxy.

    Negative pair relations act competitively: of two mutually inconsistent signals,
    the stronger one is reinforced and the weaker one is suppressed. Positive links
    provide modest contextual support.
    """
    p = s.p_base
    ctx = np.zeros(s.N)
    for i in range(s.N):
        for j in range(s.N):
            c = s.corr[i, j]
            if c < 0:
                # Competitive contextual update. If p_i < p_j, context_i decreases.
                ctx[i] += (-c) * (p[i] - p[j])
            elif c > 0:
                ctx[i] += c * (p[j] - 0.5)
    # Normalize by degree to avoid graph-size effects.
    degree = np.maximum(1.0, np.sum(np.abs(s.corr), axis=1))
    ctx /= degree
    # Use separate gain implicitly via sign.
    gain = np.where(ctx < 0, gamma_neg, gamma_pos)
    return np.clip(_sigmoid(_logit(p) + gain * ctx), 0.005, 0.995)


def expected_scores(s: Scenario, p: np.ndarray) -> np.ndarray:
    return p[:, None] * s.damage[:, None] * s.effectiveness - s.resource_cost[None, :]


def oracle_scores(s: Scenario) -> np.ndarray:
    return s.truth[:, None] * s.damage[:, None] * s.effectiveness - s.resource_cost[None, :]


# ----------------------------
# Evaluation utilities
# ----------------------------

def x_to_matrix(x: np.ndarray, N: int, M: int) -> np.ndarray:
    return np.asarray(x, dtype=np.int8).reshape(N, M)


def repair_assignment(x: np.ndarray, scores: np.ndarray) -> np.ndarray:
    """Repair to at most one resource per threat and one threat per resource."""
    N, M = scores.shape
    mat = x_to_matrix(x, N, M).copy()
    # Iteratively keep highest-score conflict until feasible.
    changed = True
    while changed:
        changed = False
        for i in range(N):
            js = np.where(mat[i] > 0)[0]
            if len(js) > 1:
                keep = js[np.argmax(scores[i, js])]
```

```
                mat[i, js] = 0
                mat[i, keep] = 1
                changed = True
        for r in range(M):
            is_ = np.where(mat[:, r] > 0)[0]
            if len(is_) > 1:
                keep = is_[np.argmax(scores[is_, r])]
                mat[is_, r] = 0
                mat[keep, r] = 1
                changed = True
    return mat.ravel()


def actual_reward(s: Scenario, x: np.ndarray) -> float:
    mat = x_to_matrix(x, s.N, s.M)
    prevented = np.sum(mat * (s.truth[:, None] * s.damage[:, None] * s.effectiveness))
    cost = np.sum(mat * s.resource_cost[None, :])
    return float(prevented - cost)


def damage_prevented_raw(s: Scenario, x: np.ndarray) -> float:
    mat = x_to_matrix(x, s.N, s.M)
    return float(np.sum(mat * (s.truth[:, None] * s.damage[:, None] * s.effectiveness)))


def decoy_waste_rate(s: Scenario, x: np.ndarray) -> float:
    mat = x_to_matrix(x, s.N, s.M)
    selected = mat.sum()
    if selected == 0:
        return 0.0
    decoy_selected = np.sum(mat * (1 - s.truth)[:, None])
    return float(decoy_selected / selected)


def selected_count(s: Scenario, x: np.ndarray) -> int:
    return int(x_to_matrix(x, s.N, s.M).sum())


def solve_oracle(s: Scenario) -> np.ndarray:
    return solve_milp_from_scores(s, oracle_scores(s))[0]


# ----------------------------
# Solvers
# ----------------------------

def solve_greedy(s: Scenario, scores: np.ndarray) -> np.ndarray:
    N, M = scores.shape
    order = np.argsort(scores.ravel())[::-1]
    used_i, used_r = set(), set()
    x = np.zeros(N * M, dtype=np.int8)
    for idx in order:
        i, r = divmod(int(idx), M)
        if scores[i, r] <= 0:
            break
        if i not in used_i and r not in used_r:
            x[idx] = 1
            used_i.add(i)
            used_r.add(r)
            if len(used_r) == M:
                break
    return x
```

```
274
275
276 def _assignment_constraints(N: int, M: int):
277     K = N * M
278     rows, cols, data = [], [], []
279     row = 0
280     # threat <= 1
281     for i in range(N):
282         for r in range(M):
283             rows.append(row); cols.append(i*M+r); data.append(1.0)
284         row += 1
285     # resource <= 1
286     for r in range(M):
287         for i in range(N):
288             rows.append(row); cols.append(i*M+r); data.append(1.0)
289         row += 1
290     A = csr_matrix((data, (rows, cols)), shape=(N+M, K))
291     lb = np.zeros(N+M)
292     ub = np.ones(N+M)
293     return LinearConstraint(A, lb, ub)
294
295
296 def solve_milp_from_scores(s: Scenario, scores: np.ndarray, time_limit: float = 2.0) -> Tuple[np.ndarray,
        Dict]:
297     c = -scores.ravel()
298     bounds = Bounds(np.zeros(s.K), np.ones(s.K))
299     integrality = np.ones(s.K, dtype=np.int8)
300     constraint = _assignment_constraints(s.N, s.M)
301     res = milp(c=c, integrality=integrality, bounds=bounds, constraints=constraint,
302                options={"time_limit": time_limit, "mip_rel_gap": 1e-8, "presolve": True})
303     if res.x is None:
304         x = solve_greedy(s, scores)
305     else:
306         x = (res.x > 0.5).astype(np.int8)
307     meta = {
308         "milp_success": bool(getattr(res, "success", False)),
309         "mip_gap": float(getattr(res, "mip_gap", np.nan)) if getattr(res, "mip_gap", None) is not None else
              np.nan,
310         "mip_node_count": int(getattr(res, "mip_node_count", 0) or 0),
311     }
312     return x, meta
313
314
315 def build_qubo(s: Scenario, scores: np.ndarray, contextual: bool = False,
316                penalty: float = 2.5, corr_penalty: float = 1.2):
317     K, N, M = s.K, s.N, s.M
318     h = -scores.ravel().astype(np.float64).copy()
319     edges: Dict[Tuple[int,int], float] = {}
320
321     def add_edge(a, b, w):
322         if a == b: return
323         if a > b: a, b = b, a
324         edges[(a,b)] = edges.get((a,b), 0.0) + float(w)
325
326     # At-most-one constraints encoded quadratically.
327     for i in range(N):
328         vars_i = [i*M+r for r in range(M)]
329         for a in range(len(vars_i)):
330             for b in range(a+1, len(vars_i)):
331                 add_edge(vars_i[a], vars_i[b], penalty)
332     for r in range(M):
333         vars_r = [i*M+r for i in range(N)]
```

```
        for a in range(len(vars_r)):
            for b in range(a+1, len(vars_r)):
                add_edge(vars_r[a], vars_r[b], penalty)

    # QAF proxy adds contextual incompatibility between negatively related threats.
    if contextual:
        neg_pairs = np.argwhere(np.triu(s.corr < -1e-12, 1))
        for i, j in neg_pairs:
            w = corr_penalty * (-s.corr[i, j])
            for r1 in range(M):
                for r2 in range(M):
                    add_edge(int(i)*M+r1, int(j)*M+r2, w)

    if edges:
        uvw = np.array([(a,b,w) for (a,b),w in edges.items()], dtype=np.float64)
        u = uvw[:,0].astype(np.int64)
        v = uvw[:,1].astype(np.int64)
        w = uvw[:,2].astype(np.float64)
    else:
        u = np.empty(0, np.int64); v=np.empty(0,np.int64); w=np.empty(0,np.float64)
    return h, u, v, w


def edges_to_csr(K: int, u: np.ndarray, v: np.ndarray, w: np.ndarray):
    deg = np.zeros(K, dtype=np.int64)
    for a,b in zip(u,v):
        deg[a]+=1; deg[b]+=1
    indptr = np.zeros(K+1, dtype=np.int64)
    np.cumsum(deg, out=indptr[1:])
    indices = np.empty(indptr[-1], dtype=np.int64)
    weights = np.empty(indptr[-1], dtype=np.float64)
    cursor = indptr[:-1].copy()
    for a,b,ww in zip(u,v,w):
        pa = cursor[a]; indices[pa]=b; weights[pa]=ww; cursor[a]+=1
        pb = cursor[b]; indices[pb]=a; weights[pb]=ww; cursor[b]+=1
    return indptr, indices, weights


@njit(cache=True)
def _sa_numba(h, indptr, indices, weights, seed, reads, sweeps, beta0, beta1, init_x):
    np.random.seed(seed)
    K = h.shape[0]
    best_x = np.zeros(K, dtype=np.int8)
    best_E = 1e300
    for read in range(reads):
        if read == 0 and init_x.shape[0] == K:
            x = init_x.copy()
        else:
            x = (np.random.random(K) < 0.08).astype(np.int8)
        local = h.copy()
        # local_i = h_i + sum_j w_ij x_j
        for i in range(K):
            sm = 0.0
            for p in range(indptr[i], indptr[i+1]):
                sm += weights[p] * x[indices[p]]
            local[i] += sm
        # energy, pair terms counted twice in x*local-h correction; calculate explicitly
        E = 0.0
        for i in range(K):
            E += h[i] * x[i]
        for i in range(K):
            if x[i] == 1:
```

```
                for p in range(indptr[i], indptr[i+1]):
                    j = indices[p]
                    if j > i and x[j] == 1:
                        E += weights[p]
        if E < best_E:
            best_E = E; best_x = x.copy()
        total_steps = max(1, sweeps * K)
        for step in range(total_steps):
            i = np.random.randint(0, K)
            frac = step / total_steps
            beta = beta0 + (beta1-beta0)*frac
            delta = (1 - 2*x[i]) * local[i]
            if delta <= 0 or np.random.random() < math.exp(-beta * delta):
                old = x[i]
                new = 1-old
                dx = new-old
                x[i] = new
                E += delta
                for p in range(indptr[i], indptr[i+1]):
                    j = indices[p]
                    local[j] += weights[p] * dx
                if E < best_E:
                    best_E = E; best_x = x.copy()
    return best_x, best_E


def solve_sa(s: Scenario, scores: np.ndarray, contextual: bool, seed: int,
             reads: int = 6, sweeps: int = 35, hybrid: bool = False):
    h,u,v,w = build_qubo(s, scores, contextual=contextual)
    indptr,indices,weights = edges_to_csr(s.K,u,v,w)
    # Use the same deterministic greedy seed for the first read in all SA variants.
    # This improves fairness: differences then come from the energy model, not initialization.
    init = solve_greedy(s, scores)
    if hybrid:
        reads = max(reads, 10); sweeps = max(sweeps, 55)
    x,E = _sa_numba(h,indptr,indices,weights,int(seed),int(reads),int(sweeps),0.2,7.0,init)
    x = repair_assignment(x, scores)
    meta = {"qubo_energy": float(E), "n_edges": int(len(w))}
    return x, meta


# ----------------------------
# Simplified particle-belief planner
# ----------------------------

def solve_particle_belief(s: Scenario, n_particles: int = 250, seed: int = 0) -> Tuple[np.ndarray, Dict]:
    """Simplified POMDP particle-belief baseline for a one-step allocation horizon.

    This is NOT full POMCP. It uses root sampling / particles to estimate contextual
    threat marginals under negative-pair exclusivity and then solves the expected
    allocation with MILP. It is included as a bridge to a future dynamic POMCP test.
    """
    rng = np.random.default_rng(seed)
    p = s.p_base
    samples = rng.random((n_particles, s.N)) < p[None,:]
    # Enforce negative-pair exclusivity probabilistically: if both sampled real, retain
    # the higher-confidence one with probability increasing with |corr|.
    neg_pairs = np.argwhere(np.triu(s.corr < -1e-12, 1))
    for i,j in neg_pairs:
        both = np.where(samples[:,i] & samples[:,j])[0]
        if len(both):
            c = -s.corr[i,j]
```

```
            for k in both:
                if rng.random() < c:
                    if p[i] >= p[j]: samples[k,j] = False
                    else: samples[k,i] = False
    p_part = samples.mean(axis=0)
    scores = expected_scores(s, p_part)
    x, meta = solve_milp_from_scores(s, scores)
    meta["particle_count"] = n_particles
    return x, meta


# ----------------------------
# Optional D-Wave Leap adapter
# ----------------------------

def try_dwave_leap(s: Scenario, scores: np.ndarray, contextual: bool = False):
    """Try true D-Wave Leap Hybrid BQM if dependency and credentials exist."""
    try:
        import dimod # type: ignore
        from dwave.system import LeapHybridSampler # type: ignore
    except Exception as e:
        return None, {"executed": False, "reason": f"Ocean unavailable: {e}"}
    try:
        h,u,v,w = build_qubo(s, scores, contextual=contextual)
        linear = {i: float(h[i]) for i in range(s.K)}
        quadratic = {(int(a),int(b)): float(ww) for a,b,ww in zip(u,v,w)}
        bqm = dimod.BinaryQuadraticModel(linear, quadratic, 0.0, dimod.BINARY)
        with LeapHybridSampler() as sampler:
            sampleset = sampler.sample(bqm)
            sample = sampleset.first.sample
        x = np.array([sample[i] for i in range(s.K)], dtype=np.int8)
        x = repair_assignment(x, scores)
        return x, {"executed": True}
    except Exception as e:
        return None, {"executed": False, "reason": str(e)}


# ----------------------------
# Benchmark runner
# ----------------------------

def model_bytes_for(method: str, s: Scenario, scores: np.ndarray, contextual=False) -> int:
    if "QUBO" in method or "Hybrid" in method:
        h,u,v,w = build_qubo(s, scores, contextual=contextual)
        return int(h.nbytes + u.nbytes + v.nbytes + w.nbytes)
    if "MILP" in method or "Particle" in method:
        # Objective + sparse assignment matrix estimate.
        return int(scores.nbytes + (s.K*2)*8 + (s.K*2)*8)
    return int(scores.nbytes)


def timed_call(fn, *args, **kwargs):
    tracemalloc.start()
    t0 = time.perf_counter_ns()
    out = fn(*args, **kwargs)
    dt_ms = (time.perf_counter_ns() - t0) / 1e6
    current, peak = tracemalloc.get_traced_memory()
    tracemalloc.stop()
    return out, dt_ms, peak / (1024**2)


def evaluate_method(method: str, s: Scenario, seed: int):
```

```
    p_qaf = qaf_context_belief(s)
    base_scores = expected_scores(s, s.p_base)
    qaf_scores = expected_scores(s, p_qaf)

    if method == "Greedy":
        (x), rt, py_mem = timed_call(solve_greedy, s, base_scores)
        meta={}; model_bytes=model_bytes_for(method,s,base_scores)
    elif method == "MILP":
        (ret), rt, py_mem = timed_call(solve_milp_from_scores, s, base_scores)
        x,meta=ret; model_bytes=model_bytes_for(method,s,base_scores)
    elif method == "QUBO-SA":
        (ret), rt, py_mem = timed_call(solve_sa, s, base_scores, False, seed, 6, 35, False)
        x,meta=ret; model_bytes=model_bytes_for(method,s,base_scores,False)
    elif method == "Context-MILP":
        (ret), rt, py_mem = timed_call(solve_milp_from_scores, s, qaf_scores)
        x,meta=ret; model_bytes=model_bytes_for(method,s,qaf_scores)
    elif method == "QAF-QUBO-SA":
        (ret), rt, py_mem = timed_call(solve_sa, s, qaf_scores, True, seed, 6, 35, False)
        x,meta=ret; model_bytes=model_bytes_for(method,s,qaf_scores,True)
    elif method == "Hybrid-BQM-local":
        (ret), rt, py_mem = timed_call(solve_sa, s, base_scores, False, seed, 10, 55, True)
        x,meta=ret; model_bytes=model_bytes_for(method,s,base_scores,False)
    elif method == "Particle-POMDP":
        (ret), rt, py_mem = timed_call(solve_particle_belief, s, 250, seed)
        x,meta=ret; model_bytes=model_bytes_for(method,s,base_scores)
    else:
        raise ValueError(method)

    oracle_x = solve_oracle(s)
    oracle_damage = damage_prevented_raw(s, oracle_x)
    oracle_reward = actual_reward(s, oracle_x)
    raw_damage = damage_prevented_raw(s, x)
    reward = actual_reward(s, x)
    quality = 100.0 * raw_damage / oracle_damage if oracle_damage > 1e-12 else 100.0
    utility_quality = 100.0 * reward / oracle_reward if oracle_reward > 1e-12 else 100.0

    return {
        "method": method,
        "N": s.N, "M": s.M, "K": s.K,
        "uncertainty": s.uncertainty, "seed": s.seed,
        "quality_pct_oracle_damage": quality,
        "utility_pct_oracle": utility_quality,
        "damage_prevented": raw_damage,
        "actual_reward": reward,
        "decoy_waste_rate": decoy_waste_rate(s, x),
        "selected": selected_count(s, x),
        "runtime_ms": rt,
        "python_peak_mb": py_mem,
        "model_kb": model_bytes/1024,
        "deadline_500ms": float(rt <= 500.0),
        "base_brier": float(np.mean((s.p_base - s.truth)**2)),
        "qaf_brier": float(np.mean((p_qaf - s.truth)**2)),
        "spoofed_decoys": int(np.sum(s.spoofed)),
        **meta,
    }


def run_benchmark(out_csv: Path, reps: int = 6):
    # Warm up numba compilation outside measurements.
    warm = generate_scenario(5,3,0.2,123)
    solve_sa(warm, expected_scores(warm,warm.p_base), False, 123, 1, 1, False)

```

```
    scales = [(5,3),(10,5),(20,5),(25,10),(50,10)] # K=15,50,100,250,500
    uncertainties = [0.10,0.35,0.60]
    methods = ["Greedy","MILP","QUBO-SA","Context-MILP","QAF-QUBO-SA","Hybrid-BQM-local","Particle-POMDP"]
    rows=[]
    total=len(scales)*len(uncertainties)*reps
    done=0
    for N,M in scales:
        for u in uncertainties:
            for rep in range(reps):
                seed = 100000 + (N*1000+M*100) + int(u*100)*10 + rep
                s = generate_scenario(N,M,u,seed)
                for mi,method in enumerate(methods):
                    row = evaluate_method(method,s,seed+mi*97)
                    rows.append(row)
                done += 1
                if done % 10 == 0:
                    print(f"Completed scenarios {done}/{total}")
    df=pd.DataFrame(rows)
    df.to_csv(out_csv,index=False)
    return df


# ----------------------------
# Plots
# ----------------------------

def make_plots(df: pd.DataFrame, figdir: Path):
    import matplotlib.pyplot as plt
    figdir.mkdir(parents=True,exist_ok=True)
    preferred=["Greedy","MILP","QUBO-SA","Context-MILP","QAF-QUBO-SA","Hybrid-BQM-local","Particle-POMDP"]

    # 1 overall quality
    g=df.groupby('method')['quality_pct_oracle_damage'].agg(['mean','std']).reindex(preferred)
    fig,ax=plt.subplots(figsize=(9,5)); ax.bar(g.index,g['mean'],yerr=g['std'],capsize=3); ax.set_ylabel('%
         del dano optimo evitado'); ax.set_title('1. Calidad global de decision'); ax.tick_params(axis='x',
        rotation=35); ax.set_ylim(0,105); fig.tight_layout(); fig.savefig(figdir/'01_calidad_global.png',
        dpi=180); plt.close(fig)

    # 2 waste rate
    g=df.groupby('method')['decoy_waste_rate'].mean().reindex(preferred)*100
    fig,ax=plt.subplots(figsize=(9,5)); ax.bar(g.index,g.values); ax.set_ylabel('Recursos asignados a
        senuelos (%)'); ax.set_title('2. Desperdicio de recursos en senuelos'); ax.tick_params(axis='x',
        rotation=35); fig.tight_layout(); fig.savefig(figdir/'02_senuelos.png',dpi=180); plt.close(fig)

    # 3 runtime vs K (median, log y)
    g=df.groupby(['method','K'])['runtime_ms'].median().reset_index()
    fig,ax=plt.subplots(figsize=(9,5.5))
    for m in preferred:
        z=g[g.method==m]; ax.plot(z.K,z.runtime_ms,marker='o',label=m)
    ax.axhline(500,ls='--',lw=1,label='500 ms'); ax.set_yscale('log'); ax.set_xlabel('Variables K'); ax.
        set_ylabel('Latencia mediana (ms, escala log)'); ax.set_title('3. Escalabilidad temporal'); ax.
        legend(fontsize=7,ncol=2); fig.tight_layout(); fig.savefig(figdir/'03_runtime_vs_k.png',dpi=180);
        plt.close(fig)

    # 4 model memory vs K
    g=df.groupby(['method','K'])['model_kb'].median().reset_index()
    fig,ax=plt.subplots(figsize=(9,5.5))
    for m in preferred:
        z=g[g.method==m]; ax.plot(z.K,z.model_kb,marker='o',label=m)
    ax.set_yscale('log'); ax.set_xlabel('Variables K'); ax.set_ylabel('Tamano estimado del modelo (KB, log)'
        ); ax.set_title('4. Escalabilidad de representacion'); ax.legend(fontsize=7,ncol=2); fig.
        tight_layout(); fig.savefig(figdir/'04_model_memory.png',dpi=180); plt.close(fig)
```

```
634
635     # 5 deadline compliance by K
636     g=df.groupby(['method','K'])['deadline_500ms'].mean().reset_index()
637     fig,ax=plt.subplots(figsize=(9,5.5))
638     for m in preferred:
639         z=g[g.method==m]; ax.plot(z.K,z.deadline_500ms*100,marker='o',label=m)
640     ax.set_xlabel('Variables K'); ax.set_ylabel('Corridas <= 500 ms (%)'); ax.set_ylim(-2,102); ax.
          set_title('5. Cumplimiento de ventana de 500 ms'); ax.legend(fontsize=7,ncol=2); fig.tight_layout()
          ; fig.savefig(figdir/'05_deadline.png',dpi=180); plt.close(fig)
641
642     # 6 quality vs uncertainty
643     g=df.groupby(['method','uncertainty'])['quality_pct_oracle_damage'].mean().reset_index()
644     fig,ax=plt.subplots(figsize=(9,5.5))
645     for m in preferred:
646         z=g[g.method==m]; ax.plot(z.uncertainty*100,z.quality_pct_oracle_damage,marker='o',label=m)
647     ax.set_xlabel('Incertidumbre sintetica (%)'); ax.set_ylabel('% del dano optimo evitado'); ax.set_title(
          '6. Robustez frente a incertidumbre'); ax.legend(fontsize=7,ncol=2); fig.tight_layout(); fig.
          savefig(figdir/'06_quality_uncertainty.png',dpi=180); plt.close(fig)
648
649     # 7 QAF relative improvement vs key baselines across uncertainty
650     agg=df.groupby(['method','uncertainty'])['quality_pct_oracle_damage'].mean().unstack('method')
651     fig,ax=plt.subplots(figsize=(9,5))
652     for b in ['Greedy','MILP','QUBO-SA','Particle-POMDP']:
653         imp=(agg['QAF-QUBO-SA']-agg[b])/agg[b]*100
654         ax.plot(agg.index*100,imp.values,marker='o',label=f'vs {b}')
655     ax.axhline(0,lw=1); ax.set_xlabel('Incertidumbre sintetica (%)'); ax.set_ylabel('Mejora relativa QAF
          (%)'); ax.set_title('7. Ventaja relativa del proxy QAF'); ax.legend(); fig.tight_layout(); fig.
          savefig(figdir/'07_qaf_improvement.png',dpi=180); plt.close(fig)
656
657     # 8 belief calibration brier
658     b=df[['seed','N','M','uncertainty','base_brier','qaf_brier']].drop_duplicates()
659     g=b.groupby('uncertainty')[['base_brier','qaf_brier']].mean()
660     fig,ax=plt.subplots(figsize=(8,5)); ax.plot(g.index*100,g.base_brier,marker='o',label='Fusion
          convencional'); ax.plot(g.index*100,g.qaf_brier,marker='o',label='Contexto QAF proxy'); ax.
          set_xlabel('Incertidumbre sintetica (%)'); ax.set_ylabel('Brier score (menor es mejor)'); ax.
          set_title('8. Calibracion de confianza'); ax.legend(); fig.tight_layout(); fig.savefig(figdir/'08
          _brier.png',dpi=180); plt.close(fig)
661
662     # 9 runtime-quality frontier at K=500
663     z=df[df.K==500].groupby('method').agg(runtime_ms=('runtime_ms','median'),quality=('
          quality_pct_oracle_damage','mean')).reindex(preferred)
664     fig,ax=plt.subplots(figsize=(8,5.5)); ax.scatter(z.runtime_ms,z.quality,s=65)
665     for m,row in z.iterrows(): ax.annotate(m,(row.runtime_ms,row.quality),xytext=(4,4),textcoords='offset
          points',fontsize=7)
666     ax.set_xscale('log'); ax.axvline(500,ls='--',lw=1); ax.set_xlabel('Latencia mediana K=500 (ms, log)');
          ax.set_ylabel('% del dano optimo evitado'); ax.set_title('9. Frontera calidad-latencia en K=500');
           fig.tight_layout(); fig.savefig(figdir/'09_frontier_k500.png',dpi=180); plt.close(fig)
667
668     # 10 benchmark architecture diagram
669     fig,ax=plt.subplots(figsize=(10,6)); ax.axis('off')
670     boxes=[(0.06,0.68,0.18,0.16,'Escenario\nsintetico'),(0.31,0.68,0.18,0.16,'Fusion de\nsensores')
          ,(0.56,0.68,0.18,0.16,'Metodos\ncomparados'),(0.81,0.68,0.14,0.16,'Decision\nx')]
671     for x,y,w,h,t in boxes:
672         ax.add_patch(plt.Rectangle((x,y),w,h,fill=False,lw=1.5)); ax.text(x+w/2,y+h/2,t,ha='center',va='
              center',fontsize=11)
673     for x1,x2 in [(0.24,0.31),(0.49,0.56),(0.74,0.81)]: ax.annotate('',xy=(x2,0.76),xytext=(x1,0.76),
          arrowprops=dict(arrowstyle='->'))
674     methods='Greedy\nMILP / HiGHS\nQUBO + SA\nContext-MILP\nQAF-QUBO + SA\nHybrid-BQM local\nParticle-POMDP'

675     ax.text(0.65,0.47,methods,ha='center',va='center',fontsize=10,bbox=dict(boxstyle='round',fill=False))
676     ax.annotate('',xy=(0.65,0.60),xytext=(0.65,0.66),arrowprops=dict(arrowstyle='->'))
```

```
    ax.text(0.28,0.30,'Ground truth reservado\npara evaluacion',ha='center',va='center',fontsize=10,bbox=dict(boxstyle='round',fill=False))
    ax.text(0.72,0.25,'Metricas: calidad, senuelos,\nlatencia, memoria, deadline,\ncalibracion y escalabilidad',ha='center',va='center',fontsize=10,bbox=dict(boxstyle='round',fill=False))
    ax.annotate('',xy=(0.62,0.31),xytext=(0.38,0.31),arrowprops=dict(arrowstyle='->'))
    ax.annotate('',xy=(0.76,0.39),xytext=(0.86,0.68),arrowprops=dict(arrowstyle='->'))
    ax.set_title('10. Diseno del banco de pruebas',fontsize=14); fig.tight_layout(); fig.savefig(figdir/'10_benchmark_diagram.png',dpi=180); plt.close(fig)


def summary_tables(df: pd.DataFrame, outdir: Path):
    methods=["Greedy","MILP","QUBO-SA","Context-MILP","QAF-QUBO-SA","Hybrid-BQM-local","Particle-POMDP"]
    overall=df.groupby('method').agg(
        quality_mean=('quality_pct_oracle_damage','mean'),
        quality_sd=('quality_pct_oracle_damage','std'),
        waste_pct=('decoy_waste_rate',lambda x:100*x.mean()),
        runtime_median_ms=('runtime_ms','median'),
        runtime_p95_ms=('runtime_ms',lambda x:np.quantile(x,0.95)),
        deadline_pct=('deadline_500ms',lambda x:100*x.mean()),
        model_kb_median=('model_kb','median'),
    ).reindex(methods)
    overall.to_csv(outdir/'summary_overall.csv')
    byk=df.groupby(['method','K']).agg(quality=('quality_pct_oracle_damage','mean'),runtime_ms=('runtime_ms','median'),deadline=('deadline_500ms','mean'),waste=('decoy_waste_rate','mean')).reset_index()
    byk.to_csv(outdir/'summary_by_k.csv',index=False)
    byu=df.groupby(['method','uncertainty']).agg(quality=('quality_pct_oracle_damage','mean'),runtime_ms=('runtime_ms','median'),waste=('decoy_waste_rate','mean')).reset_index()
    byu.to_csv(outdir/'summary_by_uncertainty.csv',index=False)
    return overall,byk,byu


def main():
    ap=argparse.ArgumentParser()
    ap.add_argument('--outdir',default='/mnt/data/qaf_benchmark')
    ap.add_argument('--reps',type=int,default=6)
    ap.add_argument('--reuse',action='store_true')
    args=ap.parse_args()
    out=Path(args.outdir); out.mkdir(parents=True,exist_ok=True); (out/'results').mkdir(exist_ok=True); (out/'figures').mkdir(exist_ok=True)
    csv=out/'results'/'benchmark_results.csv'
    if args.reuse and csv.exists(): df=pd.read_csv(csv)
    else: df=run_benchmark(csv,args.reps)
    overall,_,_=summary_tables(df,out/'results')
    make_plots(df,out/'figures')
    print('\nOVERALL SUMMARY')
    print(overall.to_string(float_format=lambda x:f'{x:,.3f}'))
    print(f'\nSaved: {csv}')

if __name__=='__main__':
    main()
```